Modulo 2

Angelica Collazo

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

##Primeros comandos de R

R es un lenguaje muy similar a MatLab y permite el trabajo con matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

Uso de datasets o base de datos internos de R

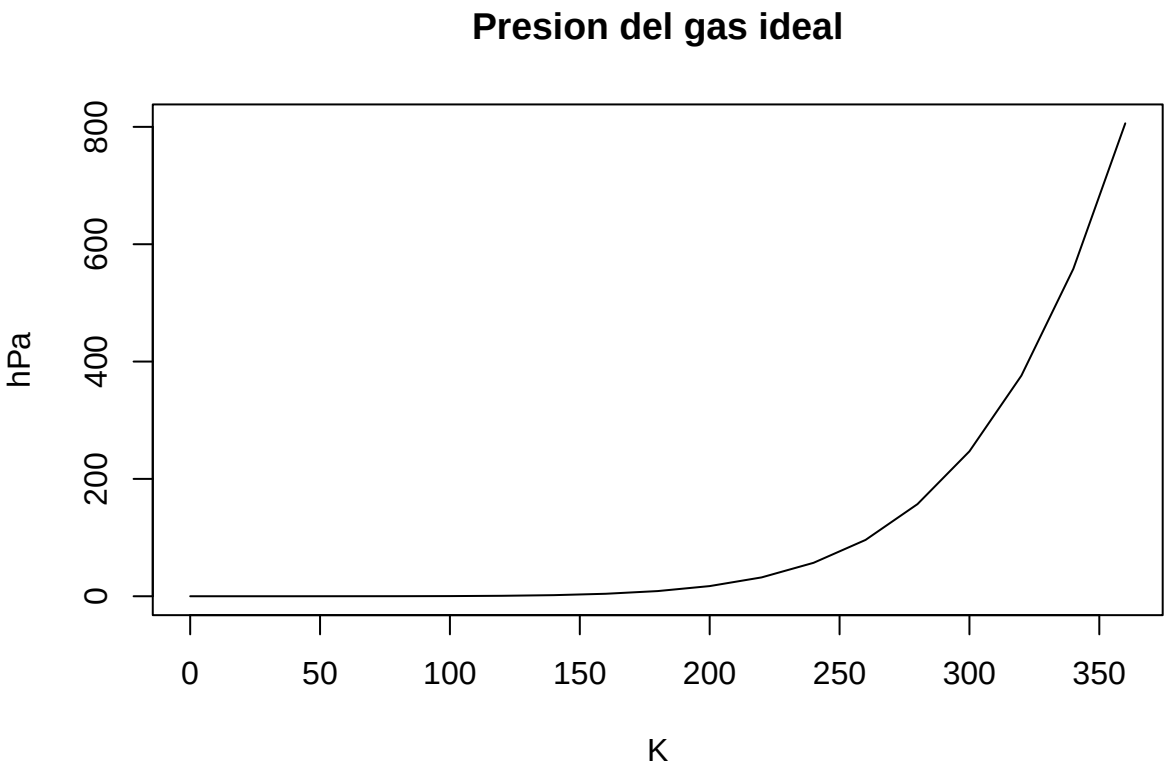
Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $, luego indicando la informacion especifica que deseamos.

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo también la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir **?plot** en la consola, lo cual abrira una ventana que explicara en detalle el comando.



Para que en grafico me aparezca la funcion con una linea, o puntos o de cualquier forma que yo quiera,escribo **type=letra** para saber que significa cada letra puedo escribir en la consola ?plot.

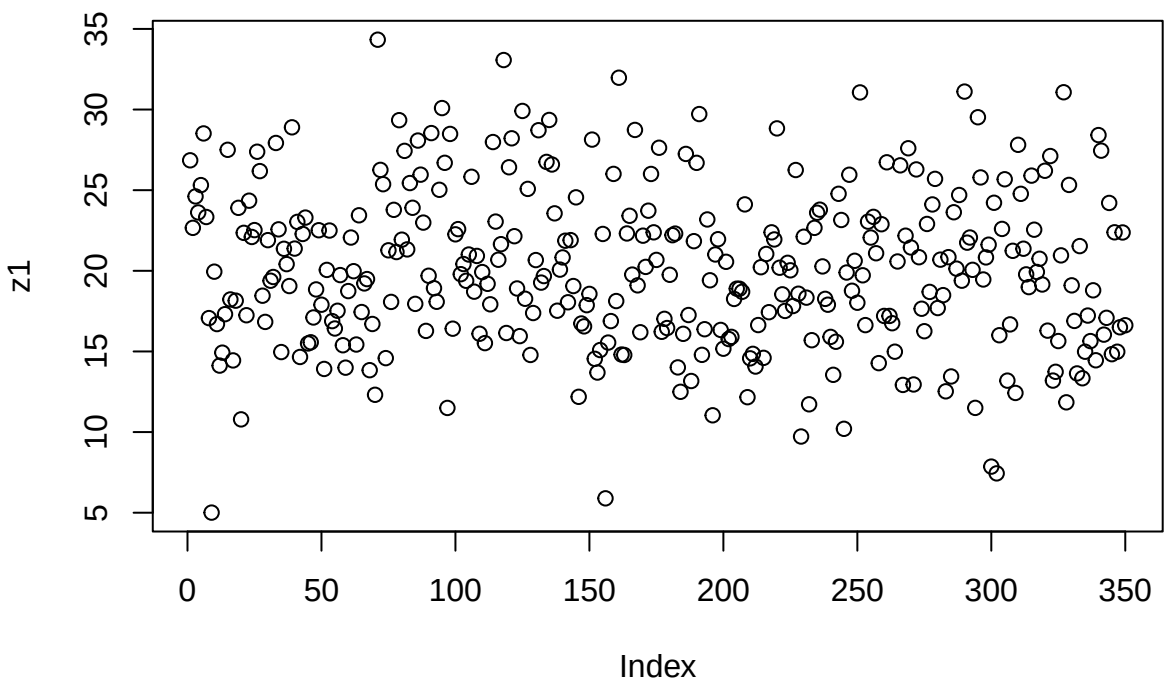
Funciones Estadisticas

```
z1<-rnorm(350,20,5)
z1
```

```
##   [1] 26.851800 22.664373 24.612988 23.620214 25.314022 28.518624 23.336180
##   [8] 17.067821  5.007284 19.946723 16.696946 14.122910 14.925470 17.311369
##  [15] 27.503113 18.223005 14.444053 18.148578 23.904707 10.788167 22.355257
##  [22] 17.245313 24.348541 22.098860 22.524122 27.385027 26.182227 18.456469
##  [29] 16.831821 21.902058 19.385672 19.615338 27.927584 22.571724 14.961859
##  [36] 21.366093 20.415557 19.055014 28.897844 21.379493 23.034134 14.656854
##  [43] 22.285415 23.299844 15.495448 15.568103 17.105933 18.845315 22.517113
##  [50] 17.897704 13.915386 20.056249 22.498158 16.880675 16.422678 17.523423
##  [57] 19.735231 15.385159 13.994403 18.743664 22.063360 19.976512 15.428613
##  [64] 23.442507 17.443424 19.218165 19.487355 13.839315 16.702034 12.312657
##  [71] 34.330777 26.259083 25.369854 14.586406 21.270572 18.072144 23.781160
##  [78] 21.163999 29.340487 21.930565 27.437984 21.328020 25.441126 23.908517
##  [85] 17.949731 28.077330 25.962575 22.989540 16.274941 19.694584 28.541093
##  [92] 18.922347 18.079784 25.023777 30.092138 26.696380 11.496647 28.487533
##  [99] 16.411608 22.261499 22.585383 19.791779 20.420907 19.381218 21.010908
## [106] 25.830262 18.706343 20.922653 16.096694 19.922193 15.516139 19.193251
## [113] 17.922733 27.984283 23.051836 20.685396 21.650915 33.069963 16.144043
## [120] 26.421125 28.216415 22.145618 18.905655 15.949682 29.913187 18.266163
## [127] 25.079375 14.782602 17.382189 20.666767 28.717392 19.268538 19.674591
## [134] 26.752480 29.354286 26.593095 23.562987 17.526438 20.074501 20.829051
## [141] 21.866158 18.048312 21.903611 19.055756 24.555922 12.187238 16.733858
## [148] 16.554743 17.870516 18.567503 28.136239 14.542349 13.691049 15.086374
```

```
## [155] 22.285166  5.891926 15.550599 16.883077 26.009684 18.125126 31.973029
## [162] 14.807770 14.781075 22.315713 23.412661 19.765084 28.735878 19.097769
## [169] 16.189655 22.179918 20.242851 23.728512 26.006272 22.385066 20.676088
## [176] 27.630215 16.218494 17.020975 16.450356 19.753771 22.208840 22.308529
## [183] 14.010793 12.497523 16.091875 27.247539 17.258399 13.167211 21.837801
## [190] 26.695723 29.718743 14.789219 16.377673 23.183872 19.412237 11.040815
## [197] 21.013551 21.958871 16.334563 15.169679 20.557931 15.769284 15.903308
## [204] 18.255589 18.892587 18.888098 18.700360 24.118793 12.162632 14.572800
## [211] 14.850751 14.063746 16.635967 20.223766 14.602047 21.044203 17.433827
## [218] 22.385981 21.944894 28.831464 20.190993 18.541423 17.513502 20.495234
## [225] 20.030938 17.817516 26.250175 18.579253  9.724486 22.113148 18.336563
## [232] 11.721972 15.696351 22.663251 23.600927 23.791167 20.272160 18.260027
## [239] 17.898511 15.902522 13.551155 15.600415 24.776031 23.150930 10.202697
## [246] 19.894293 25.958063 18.762346 20.621446 18.020705 31.062953 19.722616
## [253] 16.632747 23.048708 22.061204 23.345647 21.090238 14.275024 22.888835
## [260] 17.201460 26.727482 17.202221 16.749738 14.984348 20.578122 26.539738
## [267] 12.923281 22.177224 27.599478 21.446180 12.951373 26.286042 20.843461
## [274] 17.648417 16.254800 22.905214 18.691103 24.115000 25.700548 17.692163
## [281] 20.693081 18.492134 12.526644 20.846743 13.447983 23.631016 20.135811
## [288] 24.700937 19.387294 31.112853 21.745647 22.065411 20.061282 11.499712
## [295] 29.522397 25.790336 19.461686 20.831380 21.619898  7.859265 24.219954
## [302]  7.439706 16.003995 22.597462 25.676638 13.190329 16.677762 21.240631
## [309] 12.425829 27.813462 24.774091 21.369913 19.777460 18.989821 25.903552
## [316] 22.549980 19.928449 20.746398 19.151435 26.204235 16.290215 27.116397
## [323] 13.194053 13.738234 15.644318 20.957035 31.072245 11.844410 25.325345
## [330] 19.099178 16.892163 13.636967 21.532031 13.336735 14.982558 17.225136
## [337] 15.642227 18.791018 14.451855 28.415980 27.450958 16.042861 17.085197
## [344] 24.207876 14.831127 22.388058 14.983341 16.498232 22.376880 16.624341
```

```r
plot(z1)
```



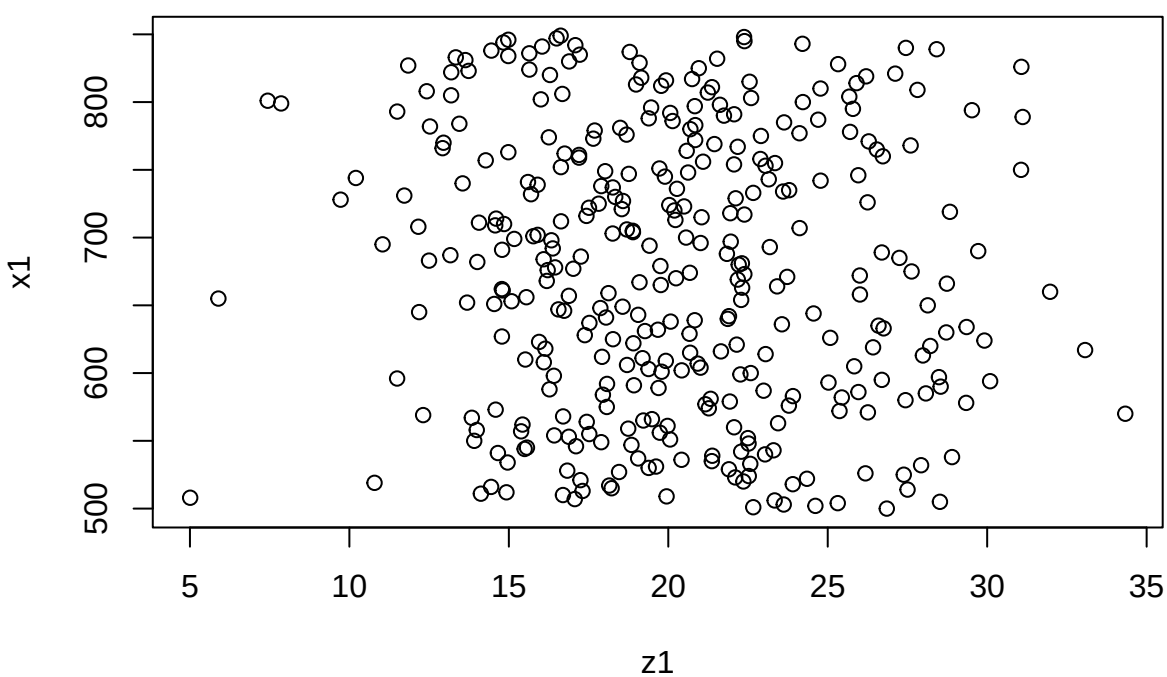

```r
w1<-length(z1)
w1
```

```
## [1] 350
```

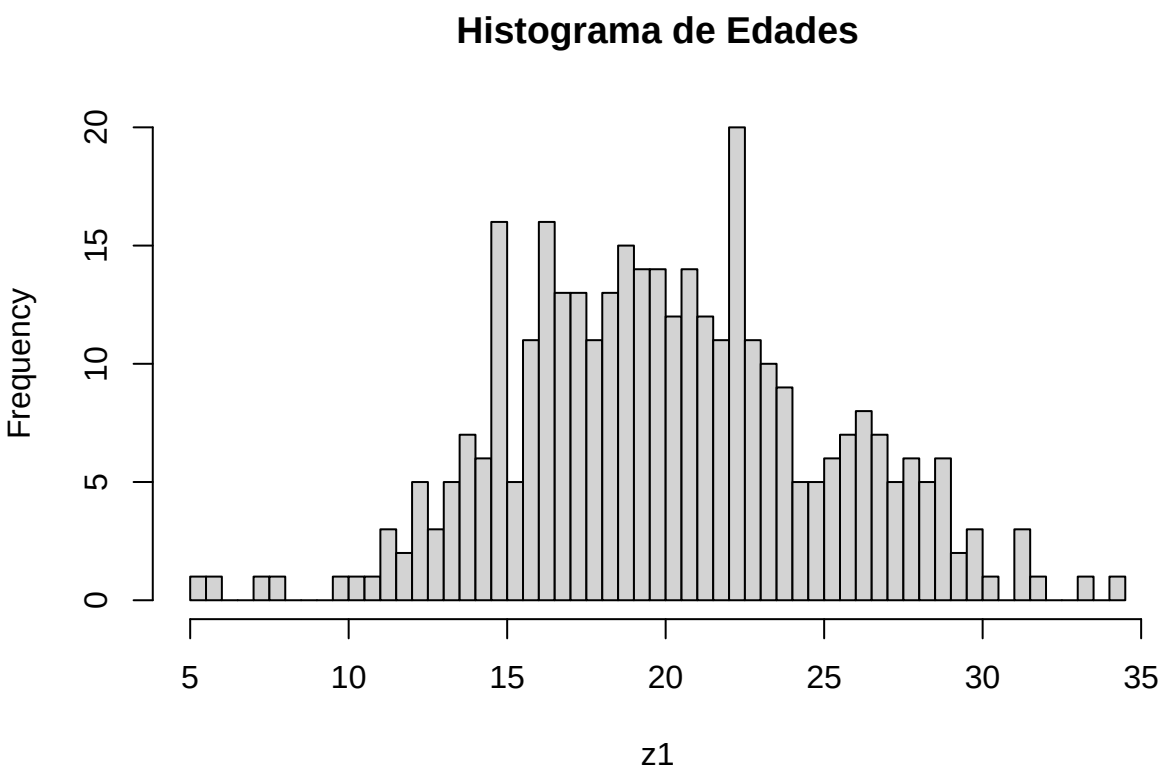
```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1,x1)
```



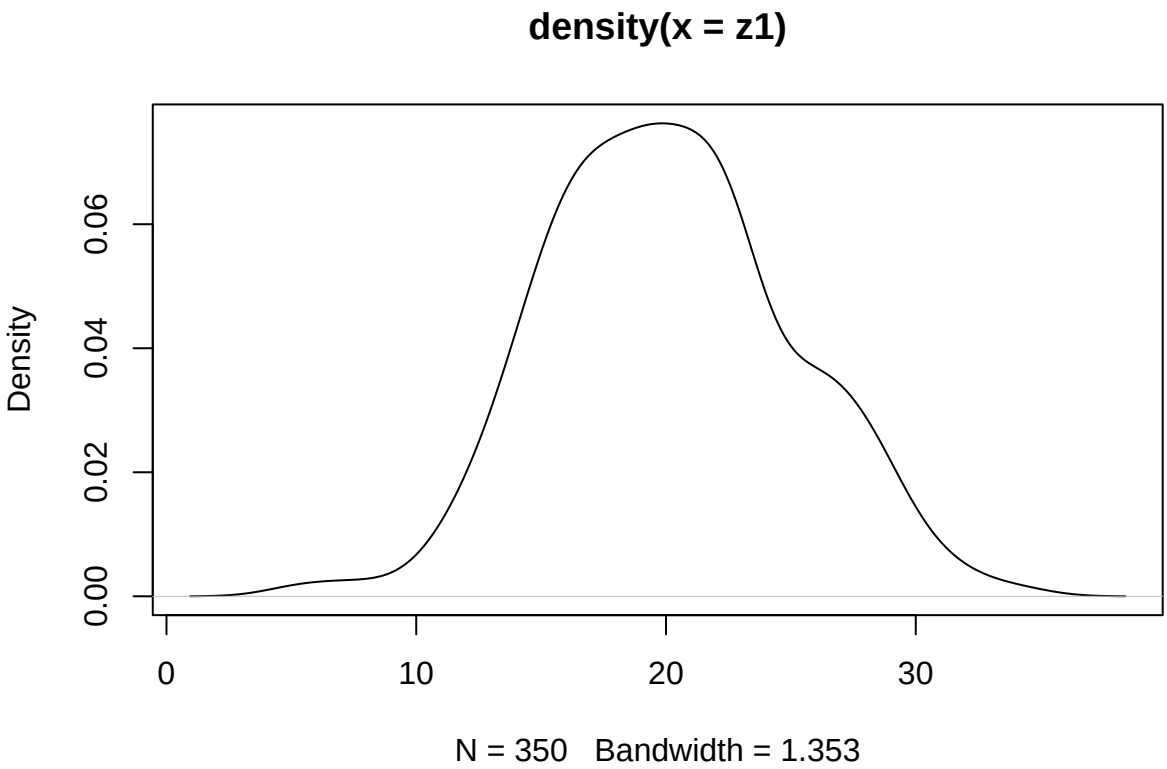

```r
hist(z1,main="Histograma de Edades",breaks=50)
```




```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.353
##
##        x                y
##  Min.   : 0.949   Min.   :9.822e-06
##  1st Qu.:10.309   1st Qu.:2.197e-03
##  Median :19.669   Median :1.413e-02
##  Mean   :19.669   Mean   :2.666e-02
##  3rd Qu.:29.029   3rd Qu.:4.962e-02
##  Max.   :38.389   Max.   :7.627e-02
```

```r
plot(density(z1))
```



El tamaño de muestra es de 350, valor promedio de 20 y un desvio estandar de 5.Si tengo alguna duda escribo en la consola **?rnorm**.

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicio 1 de algoritmo

Consigna: Las ultimas 3 cifras de mi DNI son 175,crear una variable que tenga ese numero

```
DNI=175
```

Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 al 175

```
Secuencia_dni<-(1:175)
Secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175
```

##Ejercicio 2 de algoritmo Calcular la suma de todos los valores del vector Secuencia_dni

```
Total<-0
Valor_final<-length(Secuencia_dni)
for(i in 1:175) {
```

```
Total<-Total+i
Total
}
Total
```

```
## [1] 15400
```

##Ejercicio 3 Repetir el ejercicio anterior pero en Python

```
totalp = 0
for i in range(1,176):
   totalp += i
   print(totalp)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
## 666
## 703
## 741
## 780
## 820
## 861
```

```
## 903
## 946
## 990
## 1035
## 1081
## 1128
## 1176
## 1225
## 1275
## 1326
## 1378
## 1431
## 1485
## 1540
## 1596
## 1653
## 1711
## 1770
## 1830
## 1891
## 1953
## 2016
## 2080
## 2145
## 2211
## 2278
## 2346
## 2415
## 2485
## 2556
## 2628
## 2701
## 2775
## 2850
## 2926
## 3003
## 3081
## 3160
## 3240
## 3321
## 3403
## 3486
## 3570
## 3655
## 3741
## 3828
## 3916
## 4005
## 4095
## 4186
## 4278
## 4371
## 4465
## 4560
```

```
## 4656
## 4753
## 4851
## 4950
## 5050
## 5151
## 5253
## 5356
## 5460
## 5565
## 5671
## 5778
## 5886
## 5995
## 6105
## 6216
## 6328
## 6441
## 6555
## 6670
## 6786
## 6903
## 7021
## 7140
## 7260
## 7381
## 7503
## 7626
## 7750
## 7875
## 8001
## 8128
## 8256
## 8385
## 8515
## 8646
## 8778
## 8911
## 9045
## 9180
## 9316
## 9453
## 9591
## 9730
## 9870
## 10011
## 10153
## 10296
## 10440
## 10585
## 10731
## 10878
## 11026
## 11175
```

```
## 11325
## 11476
## 11628
## 11781
## 11935
## 12090
## 12246
## 12403
## 12561
## 12720
## 12880
## 13041
## 13203
## 13366
## 13530
## 13695
## 13861
## 14028
## 14196
## 14365
## 14535
## 14706
## 14878
## 15051
## 15225
## 15400
```

##Ejercicio 4 ¿Cuanto tarda en correr el codigo del ejercicio 2?

```r
inicio<-Sys.time()
Total<-0
Valor_final<-10000000*length(Secuencia_dni)

for (i in 1:Valor_final){
    Total<-Total+i
}
Total
```

```
## [1] 1.53125e+18
```

```r
final<-Sys.time()
final-inicio
```

```
## Time difference of 33.53589 secs
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```r
system.time({
  n<-length(Secuencia_dni)

  #Aplicamos la formula de Gauss
  total_gauss <- (n*(n+1))/2
  print(total_gauss)
})
```

```
## [1] 15400
```

```
##    user  system elapsed
##       0       0       0
```

Ejercicios del profe:

1.4 Generar un vector secuencia

```r
# ---1. Generar secuencia con bucle ´for´---
#Primero inicializamos el vector
system.time({
  A<-0
  for (i in 1:50000) {
    A[i]<-(i*2)
  }
})
```

```
##    user  system elapsed
##   0.014   0.000   0.013
```

```r
# Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(B)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

1.5 Implementación de una serie Fibonachi o Fibonacci

```r
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30

# --- Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1
```

```r
  for (i in 3:n) {
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.004   0.000   0.004
```

```r
# Ver los primeros resultados
print("Primeros números de la serie:")
```

```
## [1] "Primeros números de la serie:"
```

```r
head(fib, 10)
```

```
##  [1]  0  1  1  2  3  5  8 13 21 34
```

```r
# --- Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")
```

```
## [1] "Tiempo ejecución recursiva:"
```

```r
system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})
```

```
##    user  system elapsed
##   0.861   0.000   0.879
```

1.6 Definición matemática recurrente

```r
# --- Resolver consigna con bucle 'while'

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente: f(n+1) = f(n) + f(n-1)
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.005   0.000   0.005
```

```r
# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")
```

```
## El primer número mayor a 1.000.000 es: 1346269
```

```r
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")
```

```
## Se necesitaron 31 iteraciones (pasos) para superarlo.
```

```r
# Mostramos los últimos elementos para verificar
tail(fib)
```

```
## [1]  121393  196418  317811  514229  832040 1346269
```

1.7 Ordenacion de un vector por metodo burbuja

```r
# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---

cat("Tiempo con Método Burbuja:\n")
```

```
## Tiempo con Método Burbuja:
```

```r
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})
```

```
##    user  system elapsed
##   0.080   0.000   0.081
```

```r
cat("\nTiempo con función sort() nativa de R:\n")
```

```
##
## Tiempo con función sort() nativa de R:
```

```r
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##    user  system elapsed
##       0       0       0
```

1.8 La penitencia de Newton

```r
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7

# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##    user  system elapsed
##   0.185   0.000   0.187
```

```r
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```r
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

2 Agrupacion K-mean

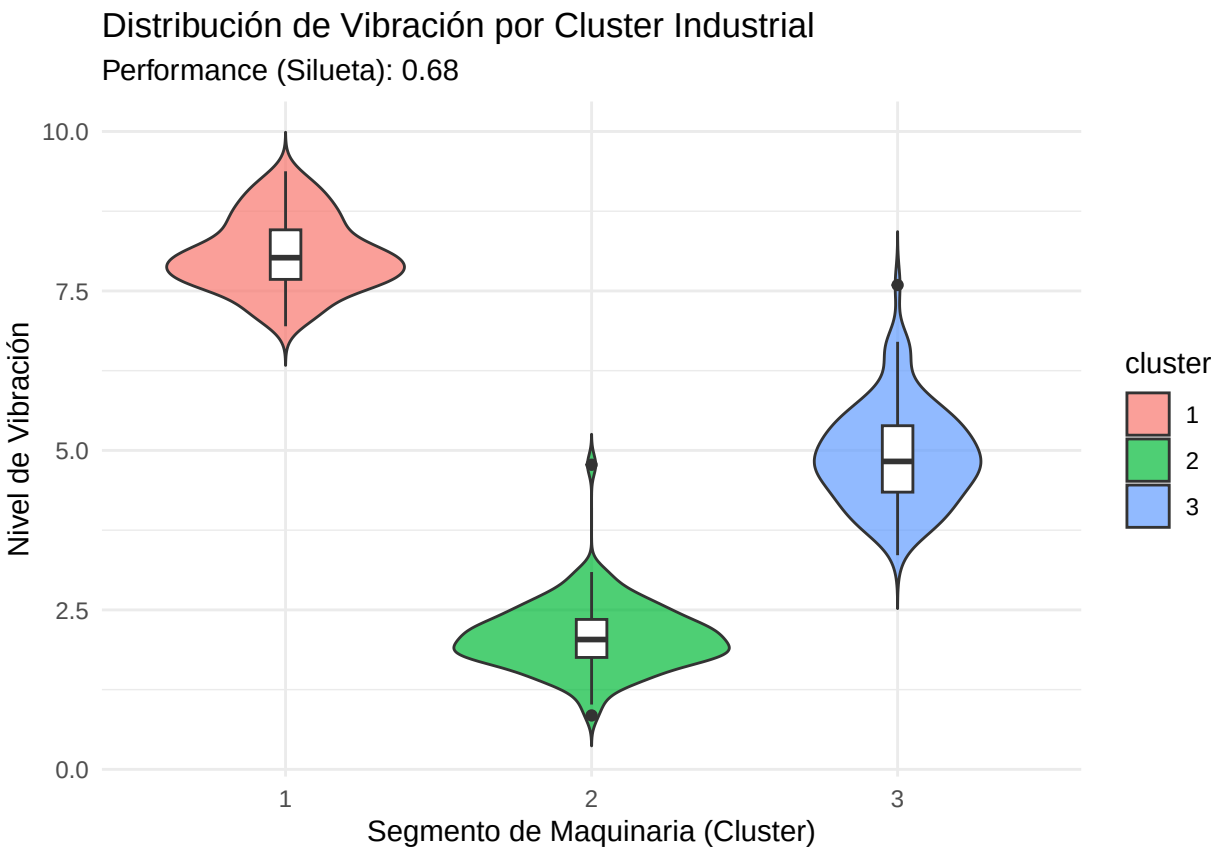
Medición de Performance: Agrupamiento K-Means en el análisis de datos industriales, no basta con agrupar los componentes; es imperativo evaluar qué tan "buenas" son esas agrupaciones. A diferencia del aprendizaje supervisado, donde comparamos predicciones contra etiquetas reales, en el Clustering medimos la calidad mediante la cohesión interna y la separación externa.1. Métricas de Evaluación InternaPara determinar si la segmentación de maquinaria tiene sentido estadístico, se utilizan principalmente dos indicadores:WSS (Within-Cluster Sum of Squares): Mide la suma de los cuadrados dentro de cada grupo. Representa la variabilidad interna. Un WSS bajo indica que los componentes dentro de un cluster son muy similares entre sí (alta cohesión).Coeficiente de Silueta (s_i): Es una medida que combina cohesión y separación. Su valor oscila entre -1 y 1:1: El componente está muy bien ubicado en su cluster.0: El componente está en el límite entre dos clusters.-1: El componente probablemente ha sido asignado al grupo incorrecto.2. El Gráfico de Violín como Herramienta de ValidaciónSi bien métricas como el "Método del Codo" nos ayudan a elegir el número óptimo de clusters (k), el gráfico de violín es esencial para la interpretación física de los resultados.Este

gráfico combina un diagrama de caja (boxplot) con una curva de densidad de probabilidad. En el contexto de mantenimiento industrial, nos permite visualizar:Distribución y Densidad: Podemos ver dónde se concentra la mayoría de los datos de sensores (vibración, temperatura) para cada grupo.Separación de Grupos: Si los "cuerpos" de los violines de diferentes clusters no se solapan significativamente, el algoritmo ha logrado identificar estados operativos claramente distintos (ej. Estado Normal vs. Falla Inminente).Detección de Anomalías: La extensión de las "colas" del violín permite identificar máquinas que, aunque pertenecen a un grupo, presentan valores extremos que requieren atención.

```r
# Librerías necesarias
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)
```

```
## El coeficiente de silueta promedio es: 0.6808356
```

```r
# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()
```



Ejercicio 7: Matrices e Inversa

En este ejercicio, crearemos una matriz cuadrada de 3x3 utilizando el valor de tu DNI (175) y otros valores para asegurar que la matriz sea invertible (que su determinante sea distinto de cero). Luego, utilizaremos la función solve que es la estándar en R para obtener la matriz inversa.

```
dni_val <- 546

datos <- c(dni_val, 10, 15,
           2, 5, 8,
           7, 1, 9)

A <- matrix(datos, nrow = 3, ncol = 3, byrow = TRUE)

cat("### Matriz Original (A):\n")
```

```
## ### Matriz Original (A):
```

```
print(A)
```

```
##      [,1] [,2] [,3]
## [1,]  546   10   15
## [2,]    2    5    8
## [3,]    7    1    9
```

```
determinante <- det(A)
cat("\nDeterminante de la matriz:", determinante, "\n")
```

```
##
## Determinante de la matriz: 20087
```

```r
if (determinante != 0) {
  A_inversa <- solve(A)
  cat("\n### Matriz Inversa (A^-1):\n")
  print(A_inversa)

  identidad <- round(A %*% A_inversa, 2)
  cat("\n### Verificación (A * A^-1 = Identidad):\n")
  print(identidad)
} else {
  cat("\nLa matriz es singular y no tiene inversa.")
}
```

```
##
## ### Matriz Inversa (A^-1):
##              [,1]         [,2]          [,3]
## [1,]  0.001841987 -0.003733758  0.0002489172
## [2,]  0.001891771  0.239408573 -0.2159605715
## [3,] -0.001642854 -0.023696918  0.1349131279
##
## ### Verificación (A * A^-1 = Identidad):
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1
```